



SIMATS ENGINEERING



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Duration : 1 Hour

Design Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

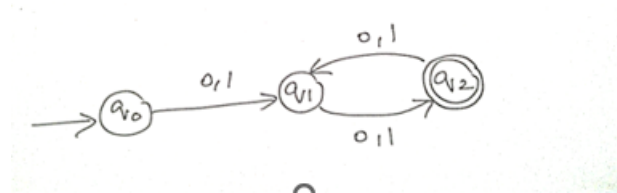


SIMATS ENGINEERING



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

1. Design the regular expression for the language represented by the given automata



ANSWER

Reading the diagram: start state q_0 , intermediate state q_1 , and final (double-circled) state q_2 , with transitions:

State	Input 0	Input 1
q_0 (start)	q_0	q_1
q_1	q_1	q_2 (final)
q_2 (final)	q_2	q_2

Step 1 - Write the Arden's Theorem equation for each state (R = start-symbol if initial + sum of incoming edges):

$$q_0 = \varepsilon + q_0.0$$

$$q_1 = q_0.1 + q_1.0$$

$$q_2 = q_1.1 + q_2.(0+1)$$

Step 2 - Solve q_0 using Arden's Theorem ($R = Q.R^*$ form, $R = \varepsilon$, $Q = 0$):

$$q_0 = \varepsilon + q_0.0 \rightarrow q_0 = 0^*$$

Step 3 - Substitute q_0 into the q_1 equation and solve using Arden's theorem ($R = 0^*.1$, $Q = 0$):

$$q_1 = 0^*.1 + q_1.0 \rightarrow q_1 = 0^*.1.0^*$$

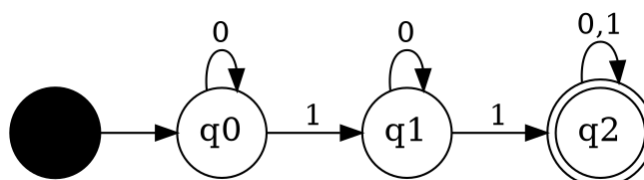
Step 4 - Substitute q_1 into the q_2 equation and solve ($R = 0^*.1.0^*.1$, $Q = (0+1)$):

$$q_2 = 0^*.1.0^*.1 + q_2.(0+1) \rightarrow q_2 = 0^*.1.0^*.1.(0+1)^*$$

Final Regular Expression $R = 0^*10^*1(0+1)^*$

(This is the set of all strings over $\{0,1\}$ that contain at least two 1's.)

Automaton used for the calculation (redrawn for clarity):



Note: if your diagram's transitions differ slightly from the table above, the same Arden's-theorem method applies - only the numeric substitutions change.



SIMATS ENGINEERING



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

2. Consider the following grammar

$$S \rightarrow aSc \mid T \mid U \mid \varepsilon$$

$$T \rightarrow bTc \mid \varepsilon$$

$$U \rightarrow aUb \mid \varepsilon$$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

ANSWER

A) Language defined by the grammar

$$T \Rightarrow b^m c^m \quad (m \geq 0) \quad \text{and} \quad U \Rightarrow a^k b^k \quad (k \geq 0)$$

$$\text{So } S \Rightarrow a^n T c^n = a^n b^m c^{m+n} \quad \dots \text{ using } T \quad (\text{call this Case 1})$$

$$\text{and } S \Rightarrow a^n U c^n = a^{n+k} b^k c^n \quad \dots \text{ using } U \quad (\text{call this Case 2})$$

$$L(G) = \{ a^i b^j c^{i+j} : i, j \geq 0 \} \cup \{ a^{i+j} b^j c^i : i, j \geq 0 \}$$

i.e. strings of the form a-block then b-block then c-block, where either $(\#c = \#a + \#b)$ or $(\#a = \#b + \#c)$. This language is context-free but not regular (it needs counting/matching of two exponents).

B) Derivation tree for aaabbc (3 a's, 2 b's, 1 c)

Check which case fits: $\#a=3, \#b=2, \#c=1$. $\#a = \#b + \#c \rightarrow 3 = 2+1 \checkmark \rightarrow$ Case 2, via U with $k=2, n=1$.

$$S \Rightarrow aUc \Rightarrow a(aUb)c \Rightarrow aa(aUb)bc \Rightarrow aa(a \cdot \varepsilon \cdot b)bc = aaabbc$$

Derivation steps:

$$S \Rightarrow aUc$$

$$\Rightarrow a(aUb)c \quad [U \rightarrow aUb]$$

$$\Rightarrow aa(aUb)bc \quad [U \rightarrow aUb]$$

$$\Rightarrow aaabbc \quad [U \rightarrow \varepsilon]$$

Tree structure (top-down): $S \rightarrow a, U, c \mid U \rightarrow a, U, b \mid U \rightarrow a, U, b \mid U \rightarrow \varepsilon$

C) Is the grammar ambiguous?

Check for overlap of Case 1 $(\#c = \#a + \#b)$ and Case 2 $(\#a = \#b + \#c)$. Solving both equations together: $c = a + b$ and $a = b + c$
 $\Rightarrow b = 0$ and $c = a$.

So every string of the form $a^n c^n$ ($b=0$) satisfies BOTH cases, meaning it has more than one derivation tree.

Example - string "ac" ($n=1$):

$$\text{Derivation 1: } S \Rightarrow aSc \Rightarrow a \cdot \varepsilon \cdot c = ac \quad [\text{using } S \rightarrow \varepsilon \text{ directly}]$$

$$\text{Derivation 2: } S \Rightarrow aSc \Rightarrow aTc \Rightarrow a \cdot \varepsilon \cdot c = ac \quad [\text{using } T \rightarrow \varepsilon]$$

$$\text{Derivation 3: } S \Rightarrow aSc \Rightarrow aUc \Rightarrow a \cdot \varepsilon \cdot c = ac \quad [\text{using } U \rightarrow \varepsilon]$$

These are 3 distinct parse trees for the same string "ac", so the grammar IS ambiguous.



SIMATS ENGINEERING



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

3. Design a non deterministic finite automata with ϵ -transitions to represent the language denoted by the regular expression $00^*1 + 11^*0$

ANSWER

Split the expression into two parts joined by union: $r1 = 00^*1$ and $r2 = 11^*0$

Build each part using Thompson's Construction, then join with a new start and new final state using ϵ -moves.

Part $r1 = 00^*1$ (states $A0 \rightarrow A1 \rightarrow A2 \rightarrow A3 \rightarrow A4 \rightarrow A5$)

From	Symbol	To
A0	0	A1
A1	ϵ	A2
A2	ϵ	A3 (loop back on 0)
A3	0	A2
A2	ϵ	A4
A4	1	A5

(A1–A2–A3 forms the 0^* block: $A1 \xrightarrow{\epsilon} A2$, $A2 \xrightarrow{\epsilon} A3$ (bypass/exit), $A3 \xrightarrow{0} A2$ loop, matching standard Kleene-star construction.)

Part $r2 = 11^*0$ (states $B0 \rightarrow B1 \rightarrow B2 \rightarrow B3 \rightarrow B4 \rightarrow B5$)

From	Symbol	To
B0	1	B1
B1	ϵ	B2
B2	ϵ	B3 (loop back on 1)
B3	1	B2
B2	ϵ	B4
B4	0	B5

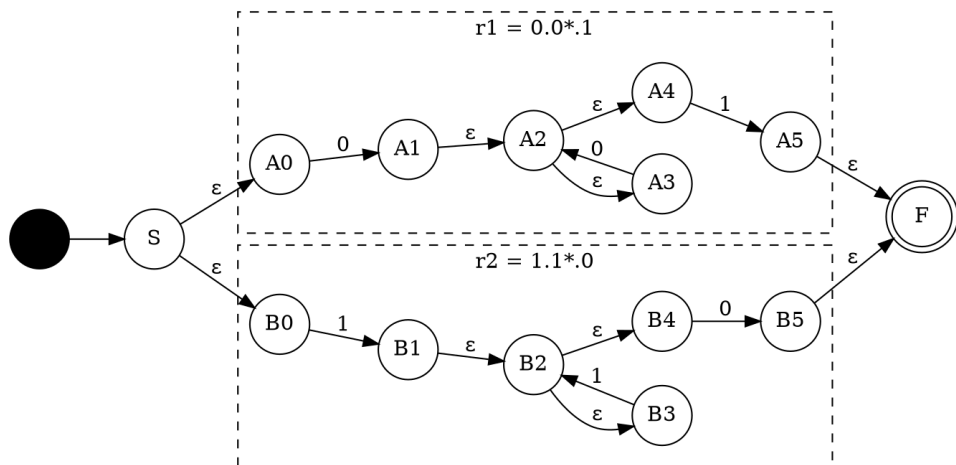
Union (final ϵ -NFA)

New start state S with: $S \xrightarrow{\epsilon} A0$ and $S \xrightarrow{\epsilon} B0$

New final state F with: $A5 \xrightarrow{\epsilon} F$ and $B5 \xrightarrow{\epsilon} F$

Start state = S, Final state = F. This ϵ -NFA accepts exactly the strings matching $00^*1 + 11^*0$ (one-or-more 0's then a 1, OR one-or-more 1's then a 0).

ϵ -NFA diagram (Thompson's Construction):





SIMATS ENGINEERING



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

4. I) Show that if L_1 and L_2 are regular then $(L_1 \cup L_2)$ is also regular

II) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

ANSWER

I) Closure of regular languages under union

Let $L_1 = L(M_1)$, where $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, and $L_2 = L(M_2)$, where $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$, both DFAs.

Construct the product automaton $M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F)$, where:

$\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$ for every a in Σ

$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$ - i.e. a pair state is final if EITHER component is final.

Claim: $L(M) = L_1 \cup L_2$.

Proof by induction on $|w|$: $\delta^*((q_1, q_2), w) = (\delta_1^*(q_1, w), \delta_2^*(q_2, w))$ for any string w (base case $w = \epsilon$ is trivial; inductive step follows directly from the definition of δ above).

So w is accepted by $M \Leftrightarrow \delta_1^*(q_1, w) \in F_1 \text{ OR } \delta_2^*(q_2, w) \in F_2 \Leftrightarrow w \in L_1 \text{ OR } w \in L_2 \Leftrightarrow w \in (L_1 \cup L_2)$.

Since M is a valid DFA with a finite number of states ($|Q_1| \times |Q_2|$), $L_1 \cup L_2$ is accepted by a DFA, hence regular. (Q.E.D.)

II) Is $\{a^p \mid p \text{ is prime}\}$ regular?

Claim: No, this language is NOT regular. Proof using the Pumping Lemma.

Assume, for contradiction, that $L = \{a^p \mid p \text{ prime}\}$ is regular. Let n be the pumping length given by the Pumping Lemma.

Since there are infinitely many primes, choose a prime $p \geq n+2$, and let $s = a^p \in L$, with $|s| = p \geq n$.

By the Pumping Lemma, $s = xyz$ with $|xy| \leq n$, $|y| \geq 1$, and $xy^i z \in L$ for all $i \geq 0$.

Let $x = a^j$, $y = a^k$ ($k \geq 1$), $z = a^{p-j-k}$. Then $|xy^i z| = p + (i-1)k$.

Choose $i = p+1$. Then the length becomes:

$$|xy^{p+1} z| = p + (p+1-1)k = p + p \cdot k = p(1+k)$$

Since $p \geq 2$ and $(1+k) \geq 2$ (as $k \geq 1$), the number $p(1+k)$ is a product of two integers each ≥ 2 , so it is COMPOSITE, never prime.

So $xy^{p+1} z = a^{p(1+k)} \notin L$, contradicting the Pumping Lemma requirement that $xy^i z \in L$ for every $i \geq 0$.

Hence the assumption is false, and $\{a^p \mid p \text{ is prime}\}$ is NOT a regular language.

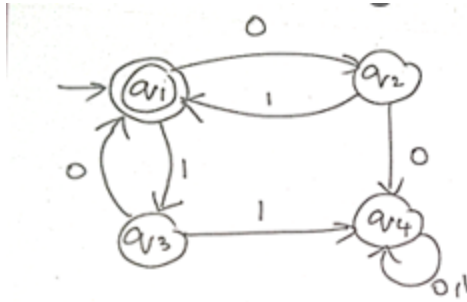


SIMATS ENGINEERING



**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

5. Design minimized automata. Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



ANSWER

Given DFA: Start state = q_0 , Final states $F = \{ q_1, q_2, q_4 \}$

State	0	1
q_0 (start)	q_3	q_1
q_1 (final)	q_2	q_5
q_2 (final)	q_2	q_5
q_3	q_0	q_4
q_4 (final)	q_2	q_5
q_5	q_5	q_5

Step 1 - Partition P_0 (final vs non-final):

$P_0 = \{ \{ q_1, q_2, q_4 \}, \{ q_0, q_3, q_5 \} \}$

Step 2 - Check block $\{ q_1, q_2, q_4 \}$:

State	$\delta(0)$	$\delta(1)$
q_1	q_2	q_5
q_2	q_2	q_5
q_4	q_2	q_5

All three go to q_2 on 0 (same block) and q_5 on 1 (same block) $\rightarrow$ not distinguishable $\rightarrow$ block stays as $\{ q_1, q_2, q_4 \}$.

Step 3 - Check block $\{ q_0, q_3, q_5 \}$:

State	$\delta(0)$	$\delta(1)$
q_0	q_3	q_1
q_3	q_0	q_4
q_5	q_5	q_5

q_0 vs q_3 : on 0 $\rightarrow q_3, q_0$ (same block); on 1 $\rightarrow q_1, q_4$ (same block, both in $\{ q_1, q_2, q_4 \}$) $\rightarrow$ NOT distinguishable.

q_0 vs q_5 : on 1 $\rightarrow q_1$ (block $\{ q_1, q_2, q_4 \}$) vs q_5 (block $\{ q_0, q_3, q_5 \}$) $\rightarrow$ DIFFERENT blocks $\rightarrow$ distinguishable.

So $\{q_0, q_3, q_5\}$ splits into $\{q_0, q_3\}$ and $\{q_5\}$.

$P1 = \{ \{q_1, q_2, q_4\}, \{q_0, q_3\}, \{q_5\} \}$

Step 4 - Re-check each block of P1 for further splitting:

$\{q_1, q_2, q_4\}$: on 0 all $\rightarrow q_2$ (same block), on 1 all $\rightarrow q_5$ (same block) $\rightarrow$ no split.

$\{q_0, q_3\}$: on 0 $\rightarrow q_3, q_0$ (same block); on 1 $\rightarrow q_1, q_4$ (same block) $\rightarrow$ no split.

$\{q_5\}$: single state, cannot split.

$P2 = P1 \rightarrow$ partitioning STOPS here.

Final equivalent states: $\{q_1, q_2, q_4\}$ merge $\rightarrow$ state B (final); $\{q_0, q_3\}$ merge $\rightarrow$ state A (start); $\{q_5\}$ stays $\rightarrow$ state C.

Minimized transition table (rename $A=\{q_0, q_3\}$, $B=\{q_1, q_2, q_4\}$, $C=\{q_5\}$):

State	0	1
A (start)	A	B
B (final)	B	C
C	C	C

Minimized automaton description (for drawing):

Start arrow $\rightarrow$ A. A is not final; B is the only final state (double circle); C is a non-final trap state.

A --0--> A (self loop), A --1--> B

B --0--> B (self loop), B --1--> C

C --0--> C, C --1--> C (self loop, trap state)

Number of states reduced from 6 (q_0-q_5) to 3 (A, B, C).

Minimized DFA diagram:

